indented template doesn't necessarily produce a perfectly indented HTML document. Clients will not be affected; well-formedness is all that matters. Your HTML could exist on a single line like `<!DOCTYPE html><html><head>...</body></html>` and all would be well.

All Twig output given in this section will have normalized spacing and indentation for the sake of not driving the typesetters crazy, and you shouldn't be concerned if your own code produces a `p` tag that is indented twice here or there, or there are extra newline characters. You can always check the validity of your output with an online HTML5 validator tool if you feel inclined, and most of those tools also implement a "pretty print" function to adjust spacing, etc.

Twig Variables

We've established that Twig is a DSL, abstracting the implementing language PHP. This begins at the data type level, with Twig offering a simplified form of the types provided by PHP. The Twig scalar types are `string`, `number` (integer or float), `boolean`, and `null`. The compound types are `iterable` (PHP arrays and iterable objects that implement `Countable Iterable` interfaces) and `object` (non-iterable objects). The data type of the underlying PHP variable determines the Twig type, and Twig is pretty fast and loose with typing: there's no explicit type casting or composite types here; PHP's type declaration system (nullable, union, disjunctive normal form, etc.) is entirely absent from Twig's template syntax.

There's no special syntax for distinguishing variable names from any other keyword in Twig, so it's a good idea to check that your variable names don't collide with existing Twig symbols. If you happened to do this, then Twig is intelligent enough to distinguish variables from other keywords and will evaluate the context of the symbol (I'll demonstrate in a minute). In most cases you will be setting up the data that your templates use before making the render call, but Twig does offer some variable manipulation allowing you to create new variables and perform operations. Use the following PHP code…

```
// index.php
require_once __DIR__ . '/vendor/autoload.php';

$loader = new \Twig\Loader\FilesystemLoader(__DIR__ . '/views');
$twig = new \Twig\Environment($loader);
```